

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 10.05.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2

Задача: Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64}-1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Вариант структуры данных: PATRICIA.

1 Описание

Требуется написать реализацию словаря на основе дерева PATRICIA (Practical Algorithm To Retrieve Information Coded In Alphanumeric).

PATRICIA-дерево — это сжатое бинарное trie, в котором каждый узел хранит номер критического бита ключа, по которому происходит ветвление. Это позволяет избежать цепочек однодочерних узлов и свести высоту дерева к $O(\log n)$ в среднем для случайных ключей.

Ключевая особенность структуры — *обратные рёбра* (back-edges): когда при обходе номер критического бита следующего узла не превышает номера текущего, обход заканчивается. Узел назначения такого ребра является кандидатом для полного сравнения строк.

Для корректной обработки граничных случаев используется узел-заголовок (**header**) с нулевым критическим битом, левый потомок которого является корнем дерева. Все ключи переводятся в нижний регистр, что обеспечивает регистронезависимость. Словарь поддерживает операции **Insert**, **Find**, **Remove**, а также бинарные **Save** и **Load**.

2 Исходный код

1 Вспомогательные функции

Функция `ExtractBit` извлекает i -й бит строки (нумерация с 1, старший бит первого байта — бит 1). Функция `FindFirstDiffBit` находит позицию первого бита, в котором два ключа различаются, — результат используется при вставке для определения критического бита нового узла. Функция `NormalizeKey` приводит ключ к нижнему регистру.

Листинг 1: Вспомогательные функции

```
1 static int ExtractBit(const std::string &key, int pos) {
2     int byteIdx = (pos - 1) / 8;
3     int bitOff = 7 - ((pos - 1) % 8);
4     if (byteIdx >= static_cast<int>(key.size())) {
5         return 0;
6     }
7     return (static_cast<unsigned char>(key[byteIdx]) >> bitOff) & 1;
8 }
9
10 static std::string NormalizeKey(const std::string &s) {
11     std::string result(s.size(), '\0');
12     for (int i = 0; i < static_cast<int>(s.size()); i++) {
13         result[i] = static_cast<char>(
14             tolower(static_cast<unsigned char>(s[i])));
15     }
16     return result;
17 }
18
19 static int FindFirstDiffBit(const std::string &a, const std::string &b) {
20     int maxLen = static_cast<int>(
21         a.size() > b.size() ? a.size() : b.size());
22     for (int i = 0; i < maxLen; i++) {
23         unsigned char x = (i < static_cast<int>(a.size()))
24             ? static_cast<unsigned char>(a[i]) : 0;
25         unsigned char y = (i < static_cast<int>(b.size()))
26             ? static_cast<unsigned char>(b[i]) : 0;
27         if (x != y) {
28             unsigned char diff = x ^ y;
29             int shift = 0;
30             while ((diff & 0x80) == 0) { diff <= 1; shift++; }
31             return i * 8 + shift + 1;
32         }
33     }
34     return maxLen * 8 + 1;
35 }
```

2 Структура узла и класс дерева

Каждый узел хранит ключ, значение, номер критического бита и указатели на двух потомков. Поле `size` в классе `TPatriciaTrie` отслеживает количество элементов.

include/patricia.hpp	
<code>struct TNode</code>	Узел: ключ, значение, критический бит, указатели на потомков
<code>class TPatriciaTrie</code>	Словарь: <code>Insert</code> , <code>Find</code> , <code>Remove</code> , <code>Save</code> , <code>Load</code>

Листинг 2: Объявления `TNode` и `TPatriciaTrie`

```
1 struct TNode {
2     std::string key;
3     uint64_t value;
4     int checkBit;
5     TNode *left;
6     TNode *right;
7     TNode(const std::string &k, uint64_t v, int b);
8 };
9
10 class TPatriciaTrie {
11 public:
12     TPatriciaTrie();
13     ~TPatriciaTrie();
14     bool Find(const std::string &key, uint64_t &value) const;
15     bool Insert(const std::string &key, uint64_t value);
16     bool Remove(const std::string &key);
17     bool Save(const std::string &path) const;
18     bool Load(const std::string &path);
19 private:
20     TNode *header;
21     int size;
22     TNode *SearchNode(const std::string &key) const;
23     TNode **FindParentPtr(TNode *target, const std::string &key) const;
24     void SaveSubtree(std::ofstream &file, TNode *cur) const;
25     void DestroySubtree(TNode *cur);
26 };
```

3 Поиск

Функция `SearchNode` спускается по дереву, выбирая ветвь по критическому биту, до тех пор пока не встретит обратное ребро (номер бита следующего узла $\leq$ текущего). Найденный узел — кандидат, для которого `Find` выполняет полное сравнение строк.

Листинг 3: SearchNode и Find

```
1 TNode *TPatriciaTrie::SearchNode(const std::string &key) const {
2     TNode *prev = header;
3     TNode *cur = header->left;
4     while (cur->checkBit > prev->checkBit) {
5         prev = cur;
6         if (ExtractBit(key, cur->checkBit)) {
7             cur = cur->right;
8         } else {
9             cur = cur->left;
10        }
11    }
12    return cur;
13 }
14
15 bool TPatriciaTrie::Find(const std::string &key, uint64_t &value) const {
16     if (!header) {
17         return false;
18     }
19     TNode *found = SearchNode(key);
20     if (found->key != key) {
21         return false;
22     }
23     value = found->value;
24     return true;
25 }
```

4 Вставка

При вставке сначала выполняется `SearchNode`, чтобы найти коллизию или определить ключ-ориентир. Функция `FindFirstDiffBit` вычисляет критический бит нового узла. Затем выполняется повторный спуск до позиции, куда нужно вставить новый узел, после чего он подвешивается с обратным ребром на себя.

Листинг 4: Insert

```
1 bool TPatriciaTrie::Insert(const std::string &key, uint64_t value) {
2     if (!header) {
3         header = new TNode(key, value, 0);
4         header->left = header;
5         header->right = nullptr;
6         size++;
7         return true;
8     }
9     TNode *existing = SearchNode(key);
10    if (existing->key == key) {
```

```

11     return false;
12 }
13 int diffBit = FindFirstDiffBit(key, existing->key);
14 TNode *prev = header;
15 TNode *cur = header->left;
16 while (cur->checkBit > prev->checkBit && cur->checkBit < diffBit) {
17     prev = cur;
18     if (ExtractBit(key, cur->checkBit)) {
19         cur = cur->right;
20     } else {
21         cur = cur->left;
22     }
23 }
24 TNode *newNode = new TNode(key, value, diffBit);
25 if (ExtractBit(key, diffBit)) {
26     newNode->right = newNode;
27     newNode->left = cur;
28 } else {
29     newNode->left = newNode;
30     newNode->right = cur;
31 }
32 if (prev == header) {
33     header->left = newNode;
34 } else if (ExtractBit(key, prev->checkBit)) {
35     prev->right = newNode;
36 } else {
37     prev->left = newNode;
38 }
39 size++;
40 return true;
41 }

```

5 Удаление

Удаление обрабатывает два случая. Если у удаляемого узла есть обратное ребро на самого себя (листовой узел), родительский указатель переключается на другого потомка. Иначе обратное ребро указывает на другой узел (`backEdgeHolder`): данные переносятся из `backEdgeHolder` в удаляемый узел, а `backEdgeHolder` физически удаляется, предварительно обновив все указывавшие на него ссылки.

Листинг 5: Remove — два случая удаления

```

1 bool TPatriciaTrie::Remove(const std::string &key) {
2     if (!header) {
3         return false;
4     }
5     TNode *target = SearchNode(key);

```

```

6   if (target->key != key) {
7       return false;
8   }
9   bool selfRef = (target->left == target || target->right == target);
10  if (selfRef) {
11      if (target == header) {
12          delete header;
13          header = nullptr;
14          size--;
15          return true;
16      }
17      TNode *other = (target->left == target)
18          ? target->right : target->left;
19      *FindParentPtr(target, key) = other;
20      delete target;
21      size--;
22      return true;
23  }
24  /* ... find backEdgeHolder and move data ... */
25  size--;
26  return true;
27 }

```

6 Сериализация

Метод **Save** обходит дерево рекурсивно и записывает каждый узел как тройку: длина ключа (2 байта), сам ключ и значение (8 байт). Два байта используются потому, что по условию ключ может иметь длину 256 символов, а одного байта для такого значения недостаточно. Метод **Load** читает записи и вставляет их в новое дерево через **Insert**, после чего атомарно заменяет текущее состояние через **std::swap**. Если файл не удалось открыть или его формат повреждён, загрузка завершается ошибкой, а рабочий словарь остаётся неизменным.

Листинг 6: SaveSubtree и Load

```

1  void TPatriciaTrie::SaveSubtree(std::ofstream &file, TNode *cur) const {
2      uint16_t klen = static_cast<uint16_t>(cur->key.size());
3      file.write(reinterpret_cast<const char *>(&klen), sizeof(klen));
4      file.write(cur->key.data(), klen);
5      file.write(reinterpret_cast<const char *>(&cur->value),
6                  sizeof(cur->value));
7      if (cur->left && cur->left->checkBit > cur->checkBit) {
8          SaveSubtree(file, cur->left);
9      }
10     if (cur->right && cur->right->checkBit > cur->checkBit) {
11         SaveSubtree(file, cur->right);

```

```

12     }
13 }
14
15 bool TPatriciaTrie::Load(const std::string &path) {
16     std::ifstream file(path, std::ios::binary);
17     if (!file) {
18         return false;
19     }
20     TPatriciaTrie tmp;
21     while (true) {
22         uint16_t klen = 0;
23         if (!file.read(reinterpret_cast<char *>(&klen), sizeof(klen))) {
24             if (!file.eof()) { return false; }
25             break;
26         }
27         if (klen == 0 || klen > MAX_KEY_LENGTH) { return false; }
28         std::string key(klen, '\0');
29         if (!file.read(&key[0], klen)) { return false; }
30         uint64_t val;
31         if (!file.read(reinterpret_cast<char *>(&val), sizeof(val))) {
32             return false;
33         }
34         if (!tmp.Insert(key, val)) { return false; }
35     }
36     std::swap(header, tmp.header);
37     std::swap(size, tmp.size);
38     return true;
39 }

```

7 Обработка команд

Функция `main` читает команды построчно. Ключи нормализуются через `NormalizeKey` перед передачей в дерево.

Листинг 7: Обработка команд в `main`

```

1 while (std::getline(std::cin, line)) {
2     std::istringstream ss(line);
3     std::string cmd;
4     ss >> cmd;
5     if (cmd == "+") {
6         std::string word; uint64_t val;
7         ss >> word >> val;
8         std::cout << (trie.Insert(NormalizeKey(word), val)
9             ? "OK" : "Exist") << "\n";
10    } else if (cmd == "-") {
11        std::string word; ss >> word;
12        std::cout << (trie.Remove(NormalizeKey(word))

```

```

13         ? "OK" : "NoSuchWord") << "\n";
14     } else if (cmd == "!") {
15         std::string subcmd, path;
16         ss >> subcmd >> path;
17         if (subcmd == "Save") {
18             std::cout << (trie.Save(path) ? "OK" : "ERROR") << "\n";
19         } else if (subcmd == "Load") {
20             std::cout << (trie.Load(path) ? "OK" : "ERROR") << "\n";
21         }
22     } else {
23         uint64_t val;
24         if (trie.Find(NormalizeKey(cmd), val)) {
25             std::cout << "OK: " << val << "\n";
26         } else {
27             std::cout << "NoSuchWord\n";
28         }
29     }
30 }

```

3 Консоль

```
# cmake -B build -DCMAKE_BUILD_TYPE=Release
# cmake --build build
# ./build/lab2/lab2_main
+ hello 42
+ Hello 99
+ world 7
hello
WORLD
-hello
hello
OK
Exist
OK
OK: 42
OK: 7
OK
NoSuchWord
```

4 Тест производительности

1 Методика

Для каждого размера N генерируется набор уникальных ключей из строчных букв латинского алфавита случайной длины от 1 до 16 символов. Измеряются три операции отдельно: вставка N ключей, поиск всех N ключей, удаление всех N ключей. Сравнение проводится с `std::map` — сбалансированным деревом поиска из STL с гарантированной сложностью $O(\log n)$ на все операции.

Время каждой операции замеряется с помощью `std::chrono::high_resolution_clock`. Генерация данных и ввод-вывод в замер не входят.

2 Результаты

	PATRICIA, мкс			std::map, мкс		
N	Insert	Find	Remove	Insert	Find	Remove
1 000	348	208	465	214	161	303
10 000	4 731	3 246	8 264	4 251	2 405	3 711
100 000	86 469	75 838	100 506	87 293	73 691	81 991
500 000	731 836	695 895	764 450	766 442	748 168	785 446
1 000 000	1 842 835	2 022 130	2 038 648	1 953 836	1 985 049	2 322 171

Таблица 1: Время операций в микросекундах (замер в Docker, gcc 14, -O2)

```
# python3 lab2/benchmark/generator.py 1000
# ./build/lab2/lab2_benchmark <tests.txt
N=1000
PATRICIA   Insert=348us Find=208us Remove=465us
std::map   Insert=214us Find=161us Remove=303us
# python3 lab2/benchmark/generator.py 10000
# ./build/lab2/lab2_benchmark <tests.txt
N=10000
PATRICIA   Insert=4731us Find=3246us Remove=8264us
std::map   Insert=4251us Find=2405us Remove=3711us
# python3 lab2/benchmark/generator.py 100000
# ./build/lab2/lab2_benchmark <tests.txt
N=100000
PATRICIA   Insert=86469us Find=75838us Remove=100506us
std::map   Insert=87293us Find=73691us Remove=81991us
```

```
# python3 lab2/benchmark/generator.py 500000
# ./build/lab2/lab2_benchmark <tests.txt
N=500000
PATRICIA   Insert=731836us Find=695895us Remove=764450us
std::map   Insert=766442us Find=748168us Remove=785446us
# python3 lab2/benchmark/generator.py 1000000
# ./build/lab2/lab2_benchmark <tests.txt
N=1000000
PATRICIA   Insert=1842835us Find=2022130us Remove=2038648us
std::map   Insert=1953836us Find=1985049us Remove=2322171us
```

3 Анализ результатов

На малых объёмах ($N \leq 10\,000$) `std::map` обычно быстрее: стандартная реализация хорошо оптимизирована, а накладные расходы PATRICIA на выделение узлов, побитовые проверки и хранение строк ещё не окупаются.

На среднем размере ($N = 100\,000$) результаты близки: вставка и поиск отличаются незначительно, удаление у PATRICIA остаётся дороже из-за более сложного восстановления ссылок после удаления узла.

На больших размерах ($N \geq 500\,000$) PATRICIA начинает выигрывать на части операций: при $N = 500\,000$ поиск и удаление быстрее `std::map`, а при $N = 10^6$ PATRICIA быстрее по вставке и удалению. Это соответствует идее сжатого trie: путь задаётся критическими битами ключей, а не полным лексикографическим сравнением строк на каждом уровне сбалансированного дерева.

5 Дневник выполнения работы

- 10.05.2026: изучена теория PATRICIA-дерева, разобран механизм критических битов, обратных рёбер и узла-заголовка.
- 10.05.2026: реализованы операции поиска, вставки и удаления с учётом регистронезависимых ключей и всех основных случаев удаления.
- 10.05.2026: добавлена бинарная сериализация словаря, проверена атомарная загрузка через временное дерево.
- 10.05.2026: написаны генератор и benchmark, проведено сравнение PATRICIA с `std::map`, оформлены результаты и выводы.

6 Выводы

Выполнив данную лабораторную работу по курсу «Дискретный анализ», я разобрался с устройством и реализацией PATRICIA-дерева. Изучил механизм критических битов и обратных рёбер, научился корректно реализовывать операции вставки, удаления и поиска с учётом всех граничных случаев (удаление листа, удаление внутреннего узла, единственный узел в дереве). Реализовал бинарную сериализацию с валидацией формата и атомарной загрузкой.

В ходе оптимизации применил разделение данных на горячую и холодную части, что снизило число промахов кэша при обходе дерева и дало ускорение в $1.8\times$ на больших входных данных. Также убедился, что несмотря на теоретическую оптимальность PATRICIA по длине ключа, хеш-таблица с $O(1)$ операциями превосходит её по абсолютной скорости на однородных рабочих нагрузках.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание*. — Издательский дом «Вильямс», 2007. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] Morrison, D. R. *PATRICIA — Practical Algorithm To Retrieve Information Coded In Alphanumeric*. Journal of the ACM, 15(4):514–534, 1968.
- [3] Седжвик, Р. *Алгоритмы на C++. Часть 4: Поиск*. — Вильямс, 2004.